

HISABDO AI

DAY 11 — SPENDING PATTERN INTELLIGENCE

Professional MVP Specification & .NET/C# Implementation Guide

1. Purpose

This module analyzes a user's HisabDo expense transactions to identify where money is being spent, which categories are highest, how spending changes between comparable periods, and which patterns appear increasing, decreasing, unusual, repeated, or recurring.

2. Business Objective

HisabDo AI should move beyond simply recording expenses and provide understandable spending intelligence. The module should answer: Where is the user spending the most? Which category changed the most? Is spending increasing or decreasing? Which expenses appear repeated or recurring?

3. Exact Task Scope

- Develop spending pattern analysis logic.
- Identify top spending categories.
- Analyze category-wise spending behavior.
- Calculate current vs previous-period changes.
- Identify increasing/decreasing spending patterns.
- Define logic for unusual/repeated spending patterns.
- Support recurring expense detection.
- Validate pattern-detection results against sample data.
- Document assumptions and edge cases.

4. Required HisabDo Data

Field	Purpose	Priority
User ID	User isolation and authorization	Required
Transaction ID	Identify each transaction	Required
Transaction Type	Separate income and expense	Required
Amount	Spending calculations	Required
Category	Category-wise analysis	Required

Date	Period comparison and trends	Required
Description	Repeated/recurring detection	Recommended

5. Analysis Period

The current period is compared with the immediately preceding period of the same duration. Example: one current month is compared with the previous month.

6. Spending Analysis Logic

- Include only valid Expense transactions belonging to the authenticated user.
- Exclude Income transactions from spending totals.
- Calculate total current-period expense.
- Group expenses by category.
- Calculate category totals and percentage of total spending.
- Sort categories by amount and return the top 5.

Category Percentage = (Category Expense / Total Expense) × 100

7. Current vs Previous Period

Absolute Change = Current Amount – Previous Amount

Percentage Change = ((Current Amount – Previous Amount) / Previous Amount) × 100

If Previous Amount = 0, percentage growth is not calculated; the category is classified as New Category when current spending is greater than zero.

8. Increasing / Decreasing / Stable Logic

Pattern	MVP Rule
Increasing	Change > +5%
Decreasing	Change < -5%
Stable	Change between -5% and +5%
New Category	Previous = 0 and Current > 0

9. Unusual Spending Pattern Logic

When sufficient historical data exists, a category may be treated as unusually high when current spending is $\geq 2 \times$ its historical average. This is an MVP configurable rule. Insufficient history must not be presented as statistically unusual.

10. Repeated Spending Pattern Logic

Repeated spending means similar expenses occur multiple times. Group transactions using normalized category and description. Repeated does not automatically mean recurring.

11. Recurring Expense Detection

- Require at least 3 comparable occurrences as an MVP confidence rule.
- Use similar category/description as supporting evidence.
- Use similar amounts as supporting evidence.
- Use approximately regular time intervals as supporting evidence.
- Return 'Recurring Candidate' rather than claiming confirmed recurrence.

12. Verified Result Structure

```
{
  "totalExpense": 150000,
  "topCategories": [
    { "category": "Food", "amount": 45000, "percentage": 30.0 }
  ],
  "categoryChanges": [
    {
      "category": "Food",
      "currentAmount": 45000,
      "previousAmount": 35000,
      "changeAmount": 10000,
      "changePercentage": 28.57,
      "trend": "Increasing"
    }
  ],
  "recurringCandidates": []
}
```

13. Complete .NET/C# Reference Implementation

```
using System;
using System.Collections.Generic;
using System.Linq;
```

```

using System.Threading;
using System.Threading.Tasks;

public sealed record SpendingTransaction(
    Guid Id, Guid UserId, string Type, decimal Amount,
    string? Category, string? Description, DateTime DateUtc);

public sealed record CategorySpending(
    string Category, decimal Amount, decimal Percentage);

public sealed record CategoryChange(
    string Category, decimal CurrentAmount, decimal
    PreviousAmount,
    decimal ChangeAmount, decimal? ChangePercentage, string
    Trend);

public sealed record RecurringCandidate(
    string Category, string? Description, decimal AverageAmount,
    int Occurrences, string Pattern);

public sealed record SpendingPatternResult(
    decimal TotalExpense,
    IReadOnlyList<CategorySpending> TopCategories,
    IReadOnlyList<CategoryChange> CategoryChanges,
    IReadOnlyList<RecurringCandidate> RecurringCandidates);

public interface ISpendingTransactionRepository
{
    Task<IReadOnlyList<SpendingTransaction>>
    GetExpensesAsync(
        Guid userId, DateTime startUtc, DateTime endUtc,
        CancellationToken cancellationToken = default);
}

public sealed class SpendingPatternIntelligenceService
{
    private const decimal TrendThresholdPercent = 5m;
    private const int TopCategoryCount = 5;
    private const int MinimumRecurringOccurrences = 3;

    private readonly ISpendingTransactionRepository _repository;

    public SpendingPatternIntelligenceService(
        ISpendingTransactionRepository repository)
    {
        _repository = repository;
    }
}

```

```

    }

    public async Task<SpendingPatternResult> AnalyzeAsync(
        Guid userId, DateTime currentStartUtc, DateTime
currentEndUtc,
        CancellationToken cancellationToken = default)
    {
        if (currentEndUtc <= currentStartUtc)
            throw new ArgumentException("End date must be after start
date.");

        var current = await _repository.GetExpensesAsync(
            userId, currentStartUtc, currentEndUtc, cancellationToken);

        var duration = currentEndUtc - currentStartUtc;
        var previousStartUtc = currentStartUtc - duration;
        var previousEndUtc = currentStartUtc;

        var previous = await _repository.GetExpensesAsync(
            userId, previousStartUtc, previousEndUtc,
cancellationToken);

        var validCurrent = current.Where(IsValidExpense).ToList();
        var validPrevious = previous.Where(IsValidExpense).ToList();

        var totalExpense = validCurrent.Sum(x => x.Amount);

        return new SpendingPatternResult(
            totalExpense,
            BuildTopCategories(validCurrent, totalExpense),
            BuildCategoryChanges(validCurrent, validPrevious),
            DetectRecurringCandidates(validCurrent));
    }

    private static bool IsValidExpense(SpendingTransaction t) =>
        t.Amount > 0 &&
        string.Equals(t.Type, "Expense",
            StringComparison.OrdinalIgnoreCase);

    private static IReadOnlyList<CategorySpending>
BuildTopCategories(
        IReadOnlyList<SpendingTransaction> transactions,
        decimal totalExpense)
    {
        return transactions
            .GroupBy(x => NormalizeCategory(x.Category))

```

```

        .Select(g => new
        {
            Category = g.Key,
            Amount = g.Sum(x => x.Amount)
        })
        .OrderByDescending(x => x.Amount)
        .Take(TopCategoryCount)
        .Select(x => new CategorySpending(
            x.Category,
            x.Amount,
            totalExpense == 0
                ? 0
                : Math.Round(x.Amount / totalExpense * 100m, 2)))
        .ToList();
    }

    private static IReadOnlyList<CategoryChange>
    BuildCategoryChanges(
        IReadOnlyList<SpendingTransaction> current,
        IReadOnlyList<SpendingTransaction> previous)
    {
        var currentByCategory = current
            .GroupBy(x => NormalizeCategory(x.Category))
            .ToDictionary(g => g.Key, g => g.Sum(x => x.Amount));

        var previousByCategory = previous
            .GroupBy(x => NormalizeCategory(x.Category))
            .ToDictionary(g => g.Key, g => g.Sum(x => x.Amount));

        var categories = currentByCategory.Keys
            .Union(previousByCategory.Keys)
            .OrderBy(x => x);

        return categories.Select(category =>
        {
            currentByCategory.TryGetValue(category, out var
currentAmount);
            previousByCategory.TryGetValue(category, out var
previousAmount);

            var changeAmount = currentAmount - previousAmount;
            decimal? changePercentage = previousAmount == 0
                ? null
                : Math.Round(changeAmount / previousAmount * 100m,
2);

```

```

        return new CategoryChange(
            category,
            currentAmount,
            previousAmount,
            changeAmount,
            changePercentage,
            ClassifyTrend(
                currentAmount,
                previousAmount,
                changePercentage));
    }).ToList();
}

private static string ClassifyTrend(
    decimal currentAmount,
    decimal previousAmount,
    decimal? changePercentage)
{
    if (previousAmount == 0 && currentAmount > 0)
        return "New Category";

    if (changePercentage > TrendThresholdPercent)
        return "Increasing";

    if (changePercentage < -TrendThresholdPercent)
        return "Decreasing";

    return "Stable";
}

private static IReadOnlyList<RecurringCandidate>
    DetectRecurringCandidates(
        IReadOnlyList<SpendingTransaction> transactions)
{
    var result = new List<RecurringCandidate>();

    var groups = transactions
        .Where(x => !string.IsNullOrEmpty(x.Description))
        .GroupBy(x => new
        {
            Category = NormalizeCategory(x.Category),
            Description = NormalizeDescription(x.Description)
        });

    foreach (var group in groups)
    {

```

```

var ordered = group.OrderBy(x => x.DateUtc).ToList();

if (ordered.Count < MinimumRecurringOccurrences)
    continue;

var intervals = ordered
    .Zip(ordered.Skip(1),
        (a, b) => (b.DateUtc - a.DateUtc).TotalDays)
    .ToList();

var averageInterval = intervals.Count == 0
    ? 0
    : intervals.Average();

var regular =
    IsNear(averageInterval, 7, 2) ||
    IsNear(averageInterval, 14, 3) ||
    IsNear(averageInterval, 30, 5) ||
    IsNear(averageInterval, 90, 10);

if (!regular)
    continue;

var averageAmount = ordered.Average(x => x.Amount);

result.Add(new RecurringCandidate(
    group.Key.Category,
    group.Key.Description,
    Math.Round(averageAmount, 2),
    ordered.Count,
    $"Approximately every {Math.Round(averageInterval, 1)}
days"));
}

return result;
}

private static bool IsNear(double value, double target, double
tolerance) =>
    Math.Abs(value - target) <= tolerance;

private static string NormalizeCategory(string? category) =>
    string.IsNullOrEmpty(category)
    ? "Uncategorized"
    : category.Trim();

```

```
private static string NormalizeDescription(string? description) =>
    string.IsNullOrEmpty(description)
        ? string.Empty
        : description.Trim().ToLowerInvariant();
}
```

14. Sample Validation Cases

- Food: current PKR 45,000 vs previous PKR 35,000 → Increasing (+28.57%).
- Transport: current PKR 9,000 vs previous PKR 10,000 → Decreasing (-10%).
- Rent: current PKR 30,000 vs previous PKR 30,000 → Stable (0%).
- New category: current PKR 8,000 vs previous PKR 0 → New Category; percentage growth is null.
- Recurring candidate: same category/description appears at least 3 times with approximately regular intervals.
- No transactions: total expense = 0 and category list is empty.
- Income-only data: income is excluded from spending analysis.
- Invalid/negative amounts: ignored by the calculation service.

15. Assumptions

- Current and previous periods have equal duration.
- Transaction Type identifies Expense separately from Income.
- Missing categories are grouped as Uncategorized.
- 5% trend threshold is an MVP rule and can be calibrated with real HisabDo data.
- Recurring detection is a candidate detector, not a confirmed subscription detector.
- Unusual-pattern detection requires sufficient historical observations.
- Actual HisabDo entities, DbContext, repository, authentication and database schema must be mapped before production merge.

16. Edge Cases

- No current-period transactions.
- No previous-period transactions.
- Previous category amount is zero.
- Total expense is zero.
- Missing category or description.
- Zero or negative amount.
- Duplicate transactions.
- Irregular recurring intervals.
- Large one-time expenses.
- Insufficient history.
- Multiple users / authorization boundaries.

17. Validation Checklist

- ☐ Total expense matches manual calculation.
- ☐ Top categories are correctly ordered.
- ☐ Category percentages are correct.
- ☐ Current vs previous changes are correct.
- ☐ Trend boundaries are correct.
- ☐ New categories are handled safely.
- ☐ Recurring candidates require sufficient evidence.
- ☐ User data is isolated.
- ☐ Empty and insufficient-history cases are safe.

18. Team Handoff

Jaffer — Backend/Data Integration: implement the calculation service against the actual HisabDo .NET schema and repository.

Omesha — AI/ML: validate pattern rules, thresholds, assumptions and sample datasets.

Aarti — AI Explanation: explain only verified spending-pattern results.

Laiba — Frontend: display total spending, top categories, trends, recurring candidates and AI insights.

Taha — Team Lead: review requirements, formulas, implementation, validation and final acceptance.

19. Final Definition

HisabDo Spending Pattern Intelligence analyzes verified expense transactions to show total spending, top categories, category-wise behavior, period-over-period changes, increasing/decreasing patterns, unusual/repeated patterns and possible recurring expenses. The .NET backend performs deterministic calculations; AI can later explain the verified results without inventing financial facts.